

Network Flow Capacity Toolkit

Chapter 1

Network Flow Capacity Toolkit

1.1 Introduction

The reachability toolkit expresses system performance in terms of network survivability, quantifying the probability that at least one active source-to-sink path exists. In many infrastructure systems, however, this binary view gives only a partial picture of operational performance.

In systems that carry flow, the operational value depends not only on *whether* a path exists but on *how much* throughput the network can sustain. For example, in a resource distribution network every source-to-sink path may satisfy the reachability threshold, yet degraded edge capacities can still leave the network unable to meet demand. Structural connectivity measures alone cannot capture this distinction.

To address this gap, the network flow toolkit adopts a capacity-based reasoning model where system performance is expressed in terms of flow quantities rather than connectivity alone. All algorithms compute exact optimisation outputs for fixed-capacity instances and produce reliability-relevant structural indicators. The rationale for exact deterministic analysis is deliberate: even networks subject to stochastic failure require a sound structural baseline before event probabilities can be interpreted meaningfully.

Given a capacitated directed acyclic graph, the toolkit addresses six operational queries: (i) what maximum throughput can the network deliver under current capacities, (ii) which cut structures form binding bottlenecks, (iii) which component failures or perturbations most affect throughput, (iv) what degradation or upgrade threshold is required to maintain a target demand, (v) how much edge or node redundancy exists in path-disjoint terms, and (vi) which components are single points of failure. The methodological contribution is therefore not a single algorithm but an integrated analysis layer that maps each of these questions to exact network-flow outputs.

1.2 Mathematical Model

1.2.1 Capacitated network definition

Definition 1.1. *Let a directed capacitated network be*

$$G = (V, E, c, S, T),$$

where V is a finite node set, $E \subseteq V \times V$ is the directed edge set, $c : E \rightarrow \mathbb{R}_{\geq 0} \cup \{\infty\}$ is an edge-capacity function, and $S, T \subseteq V$ are source and sink sets. A feasible flow is a mapping $f : E \rightarrow \mathbb{R}_{\geq 0}$ satisfying

$$0 \leq f(u, v) \leq c(u, v), \quad \forall (u, v) \in E,$$

and conservation at internal nodes:

$$\sum_{u:(u,v) \in E} f(u, v) = \sum_{w:(v,w) \in E} f(v, w), \quad \forall v \in V \setminus (S \cup T).$$

In this framework, the directed acyclic graph property is treated as an upstream input-processing invariant. That is, acyclicity is guaranteed by preprocessing and then assumed in this layer rather than re-tested at every analysis call. This design prevents duplicated checks across modules while preserving a clear contract between ingestion and analysis.

For multi-source operation, the total flow value is represented by aggregate source outflow:

$$|f| = \sum_{s \in S} \sum_{w:(s,w) \in E} f(s, w).$$

The same quantity is equal to aggregate sink inflow under conservation.

1.2.2 Max-flow optimization problem

The baseline throughput problem is

$$\max_f |f|,$$

subject to capacity feasibility and conservation. In engineering terms, this gives the maximum deliverable steady-state transfer rate under current capacities. In this chapter, all subsequent bottleneck and impact analyses are rooted in this solved baseline.

For computational implementation, multi-source/multi-sink cases are reduced exactly to a single source-single sink instance by super-node augmentation. A super-source s^* is connected to all sources, and all sinks connect to super-sink t^* with sufficiently large ca-

capacities. This transformation preserves feasible-flow correspondence and objective value.

1.2.3 Max-flow/min-cut theorem and interpretation

Theorem 1.1 (Max-flow/min-cut). *For a capacitated directed network with source s and sink t ,*

$$\max |f| = \min_{(S,T)} \text{cap}(S, T),$$

where

$$\text{cap}(S, T) = \sum_{(u,v) \in E: u \in S, v \in T} c(u, v).$$

This theorem is the central methodological anchor of the chapter (Ford and Fulkerson, 1956). Operationally, it means throughput and bottleneck are dual views of the same solved instance: maximizing admissible transfer is equivalent to finding the least-capacity separating barrier. In reliability interpretation, minimum cuts identify structural vulnerability surfaces that can enforce throughput limits.

The implementation ties theorem to verification by checking solved flow value against solved cut capacity under numerical tolerance. This dual consistency is critical because derived analyses in later sections rely on both primal flow state and cut structure extracted from the same solve.

1.2.4 Auxiliary theorems supporting downstream analyses

Several additional results underpin the derived analyses presented later in this chapter. Each is stated here for reference and linked to its computational and reliability role.

Integrality theorem (Ford and Fulkerson, 1962): if all edge capacities are integers, there exists an optimal flow in which all edge-flow values are integers. This result is essential for connectivity analyses, where unit-capacity constructions are used and the integer-valued max-flow is interpreted directly as a count of edge-disjoint or node-disjoint paths.

Menger edge theorem (Menger, 1927): the maximum number of edge-disjoint directed $s \rightarrow t$ paths equals the minimum cardinality of an s - t edge cut. Under unit-capacity constructions, the integer max-flow therefore equals the edge-disjoint path count, providing an exact flow-based route-independence interpretation for redundancy indicators.

Menger node theorem (Menger, 1927): the maximum number of internally node-disjoint $s \rightarrow t$ paths equals the minimum internal node-cut size. Combined with the connectivity ordering $\kappa \leq \lambda \leq \delta$ (Whitney, 1932), this grounds the node-connectivity computation in the same flow framework as edge connectivity.

Min-cut lattice structure (Picard and Queyranne, 1982): minimum cuts are not unique in general; they form a lattice characterised by residual reachability bounds.

Specifically, let S^* denote nodes reachable from the source in the residual graph after an optimal solve, and let S^{**} denote the complement of nodes backward-reachable from the sink. Every minimum cut partition S satisfies $S^* \subseteq S \subseteq S^{**}$. This lattice structure enables exact identification of which edges lie in *at least one* minimum cut and which edges lie in *every* minimum cut, supporting efficient single-point-of-failure detection and bounded enumeration of alternative bottleneck structures from a single baseline solve.

Node-splitting bijection: for any node v with throughput limit $c(v)$, replace v by an in-node v_{in} and out-node v_{out} connected by an internal edge of capacity $c(v)$, with all original incoming edges remapped to v_{in} and outgoing edges from v_{out} . The resulting edge-capacitated graph has a bijection between its feasible flows and the original node-capacitated feasible flows, with objective value preserved. This transformation allows all node-capacitated analyses to reuse edge-capacitated solvers without changing optimization semantics.

Flow decomposition on DAGs: any feasible flow f on a DAG can be represented as a finite weighted sum of source-to-sink path flows,

$$f = \sum_i f_i \mathbf{1}_{P_i}, \quad f_i > 0,$$

with exact per-edge accounting $f(e) = \sum_{i:e \in P_i} f_i$ and $\sum_i f_i = |f|$. This decomposition is used for throughput-allocation interpretation and route-utilisation analysis, and is conceptually distinct from structural path enumeration as discussed in Section 1.4.

These results are not included as isolated theoretical statements; each is directly linked to a computational module and a corresponding reliability interpretation in the sections that follow. Primary references for the full theory are Ahuja, Magnanti, and Orlin (1993) and the specific sources cited above.

1.3 Algorithms and Computational Strategy

1.3.1 Residual-graph framework

All implemented max-flow variants operate on residual networks. Forward residual edges encode remaining admissible capacity $c(u, v) - f(u, v)$, while backward residual edges encode cancelable flow $f(u, v)$. Augmenting or push/relabel operations update this residual state until no further admissible throughput increase exists. The no-augmenting-path condition underpins optimality in augmenting-path formulations.

This shared residual framework provides methodological consistency: different algorithms can be swapped while preserving input-output semantics and downstream analysis interfaces.

1.3.2 Implemented solver family

Three algorithm families are implemented under a common input-output contract:

1. **Edmonds–Karp** (Edmonds and Karp, 1972): breadth-first shortest-path augmenting-path selection. Worst-case complexity $O(VE^2)$. The use of BFS shortest paths bounds the number of augmenting phases by monotone distance growth on saturated critical edges.
2. **Dinic** (Dinic, 1970): repeated level-graph construction via BFS followed by depth-first blocking-flow phases with pointer-advance optimisation. Standard worst-case bound is $O(V^2E)$ for general directed graphs; tighter bounds have been established for restricted graph classes at the theory level, though any such improvement in practice depends on implementation specifics and graph structure. Dinic is the **default algorithm** throughout the framework. This default reflects an implementation design choice for this reliability pipeline, not a benchmark-supported claim of universal empirical superiority over other algorithm families or over external packages.
3. **Push–relabel** (Goldberg and Tarjan, 1988): preflow-based approach alternating admissible push operations with node relabelling. The implementation initialises source height to $|V|$, dispatches initial source excess, and selects the minimum active node at each discharge step. The active-node selection discipline used here is correct and produces exact results, but the specific asymptotic bound depends on queue discipline; no specific superior worst-case bound is claimed for this implementation configuration beyond correctness.

All three solvers share the residual-graph contract described above. Algorithm selection is centralised via symbolic dispatch (`:edmonds_karp`, `:dinic`, `:push_relabel`), preserving a stable public result contract while allowing solver substitution without changing analysis-call signatures.

1.3.3 Exact multi-source/multi-sink reduction

Given source set S and sink set T , define augmented network $G' = (V', E')$ with

$$V' = V \cup \{s^*, t^*\},$$

and edges

$$(s^*, s), s \in S, \quad (t, t^*), t \in T.$$

Augmentation edges are assigned infinite (or sufficiently large) capacity so that they never form the binding constraint. With this construction, max-flow in G' equals aggregate

multi-terminal throughput in G , and any feasible flow in G' corresponds to a feasible multi-source/multi-sink flow in G . The transformation is exact and avoids decomposition approximations that would arise if terminals were handled by flow splitting or iterative allocation.

In implementation, super-node identifiers are constructed as $s^* = \min(V) - 1$ and $t^* = \min(V) - 2$, ensuring that no collision with existing node indices occurs. An explicit underflow guard is applied to protect against pathological cases in which node identifiers approach the minimum representable 64-bit integer value, preventing arithmetic overflow in index arithmetic.

1.3.4 Directed global min-cut procedure

Directed global minimum cut is computed via a two-pass fixed-terminal strategy. In the first pass, a single node is fixed as source and max-flow is solved to every other node in turn as sink. In the second pass, a single node is fixed as sink and max-flow is solved from every other node as source. The global minimum cut capacity is the minimum over all $2(|V| - 1)$ pairwise solves, with the corresponding cut partition retained. This procedure is exact for the directed global min-cut objective.

The total solve count of $2(|V| - 1)$ implies an $O(|V|)$ multiplier on per-solve cost and should be taken into account when deploying this routine on large sparse networks. Faster specialised algorithms for directed global minimum cut have been established in the literature, including the method of Hao and Orlin (1994), which achieves asymptotically tighter bounds using parametric preflow techniques (Journal of Algorithms, 17(3):424–446; DOI: 10.1006/jagm.1994.1043). The present implementation prioritises transparent integration and state-rich compatibility with downstream post-analysis workflows over asymptotic optimality on large instances.

1.4 Derived Reliability Analyses

1.4.1 Sensitivity analysis

Sensitivity analysis quantifies how throughput responds to local capacity variation. Three structural metrics are computed. The *zeroing-impact ranking* measures the throughput drop when an edge capacity is set to zero:

$$\Delta(e) = F(c) - F(c_e \leftarrow 0).$$

The *finite-difference marginal gain* estimates the increase in max-flow under a small positive increment δ in edge capacity:

$$m_\delta(e) = \frac{F(c_e + \delta) - F(c)}{\delta}, \quad \delta > 0.$$

The *range-style flow-analogue importance score* (Birnbaum, 1969) captures the full throughput difference between zero-capacity and effectively-infinite-capacity states:

$$I_B(e) = F(c_e \leftarrow \infty) - F(c_e \leftarrow 0).$$

All three metrics are evaluated only for edges in the baseline min-cut candidate set (specifically, saturated edges identified as lying in at least one minimum cut). Edges not in any minimum cut cannot increase max-flow when their capacity is raised, since they lie strictly inside the source or sink partition of every residual cut; their sensitivity values are assigned zero without a re-solve. This candidate-set restriction is a principled computational reduction grounded in the min-cut structure, not an arbitrary heuristic.

A critical reporting qualifier applies: computed values are exact *for the evaluated candidate set*. The study is not globally exhaustive over all edges unless full-space evaluation is explicitly performed. This scope boundary must accompany any result reporting to preserve methodological honesty while still providing actionable engineering ranking signals.

1.4.2 Failure impact analysis

Failure impact analysis performs perturb-and-resolve experiments to measure throughput loss under discrete component removal, complementing the marginal focus of sensitivity analysis. For single-edge studies, each tested edge is removed entirely (capacity set to zero), and the exact max-flow is recomputed to obtain the post-failure throughput. For k -edge simultaneous failure studies, combinations of edges are zeroed together and throughput loss $\Delta(E') = F(c) - F(c_e \leftarrow 0 \ \forall e \in E')$ is computed per combination, with combination enumeration bounded by a user-specified limit to control exponential growth.

The candidate set for single-edge evaluation is restricted to saturated edges identified as lying in at least one minimum cut, since any edge strictly inside a partition cannot affect the current binding constraint under removal. Each evaluated perturbation is solved to exact optimality; reported throughput losses are therefore exact for the tested scenarios. However, the coverage scope is controlled by both candidate filtering and, for k -edge studies, combination enumeration limits. Conclusions should distinguish between *scenario-exact* results and *globally exhaustive* results, as the latter requires evaluation over all candidate subsets within the combination space.

Capacity degradation scenarios, where each edge capacity is reduced by a specified factor or override value rather than fully zeroed, are handled as individual exact re-solves with no candidate pruning, since each scenario represents a complete re-specification of the network rather than a perturbation from a common baseline cut structure.

1.4.3 Single-point-of-failure analysis

A single point of failure (SPOF) is a component whose loss causes complete throughput collapse for the defined source-sink objective. The analysis is grounded in the min-cut lattice structure described in Section 2. Specifically, let S^* denote the set of nodes reachable from the source in the residual graph after an optimal solve, and let S^{**} denote the complement of nodes backward-reachable from the sink restricted to original network nodes. An edge (u, v) lies in *every* minimum cut if and only if it is saturated, $u \in S^*$, and $v \notin S^{**}$. Such edges are structural SPOFs: their removal necessarily destroys all minimum cuts simultaneously, collapsing throughput to zero. This characterisation avoids additional solver calls beyond the baseline solve, making SPOF identification computationally negligible once the residual state is available.

Node SPOF variants combine reachability-based checks and node-capacitated evaluations. A node on every source-to-sink path whose removal disconnects source from sink is a structural node SPOF; this is identified by reachability analysis under node removal with candidate prefiltering to nodes on at least one source-to-sink path. Both edge and node SPOF outputs are high-value for infrastructure design because they identify no-redundancy choke points where minor disruptions can induce total service failure.

1.4.4 Path-based analysis: enumeration vs decomposition

Two path concepts are separated intentionally and should not be conflated:

1. **Structural path enumeration:** all feasible source-to-sink paths under the network topology, independent of solved flow values and bounded by a user-specified path limit.
2. **Flow decomposition:** an additive decomposition of the solved edge-flow mapping into a finite set of source-to-sink path flows $\{(P_i, f_i)\}$ satisfying $f(e) = \sum_{i:e \in P_i} f_i$ and $\sum_i f_i = |f|$. This decomposition reconstructs solved edge flows exactly and represents actual throughput allocation.

The first provides route-diversity information for structural analysis; the second provides throughput-allocation information tied to the solved optimal flow. Conflating them can produce incorrect reliability interpretations: a network may have many structural

source-to-sink paths while actual throughput concentrates on a small number of bottlenecked corridors. Path bottleneck values computed from structural enumeration without reference to solved flows are not additive in general and should not be summed to approximate throughput. The flow decomposition, by contrast, is validated by exact additive consistency with solved edge flows.

1.4.5 Parametric threshold, margin analysis, and node-capacitated flow

For a target throughput T , parametric threshold analysis determines how much component capacity can degrade before the network violates T , or how much must be upgraded to achieve T . The foundation is the monotone non-decreasing, concave, and piecewise-linear behaviour of max-flow $F(c_e)$ as a function of a single edge capacity with all other capacities fixed (Gallo, Grigoriadis, and Tarjan, 1989). On each linear segment between breakpoints c_{lo} and c_{hi} , the slope is

$$a = \frac{F(c_{hi}) - F(c_{lo})}{c_{hi} - c_{lo}},$$

and the threshold solving $F(c_e^*) = T$ is

$$c_e^* = c_{lo} + \frac{T - F(c_{lo})}{a}.$$

The *degradation threshold* $c_e^*(\text{degrade})$ is found by recursive partition localisation working downward from current capacity, with direct resolution when boundary partitions match. The *upgrade threshold* uses an upward doubling search to bracket the feasible range; if the max-flow function reaches a plateau under repeated doubling (indicating a non-binding bottleneck constraint on that edge), the function returns an upgrade-ineffective flag with a sentinel value of infinity, signalling that upgrading the target edge cannot increase throughput because the binding bottleneck lies elsewhere.

The deterministic reliability margin for a target T is

$$M_e = c(e) - c_e^*(\text{degrade}),$$

where $c(e)$ is current capacity and c_e^* is the degradation threshold. A large positive margin indicates robust operational slack; a small or negative margin indicates proximity to demand failure under degradation.

Node-capacitated flow is modelled via the exact node-splitting bijection described in Section 2. In this transformation, each node v with capacity $c(v)$ is replaced by (v_{in}, v_{out}) connected by an internal edge of capacity $c(v)$, with an ID convention using $v_{in} = 2v$ and $v_{out} = 2v + 1$ and explicit overflow and collision guards. All edge-capacitated analyses,

including threshold, sensitivity, and SPOF detection, can then be applied to the split graph with results interpreted back in terms of original node constraints. Node-level SPOF detection under this formulation combines re-solves for finite-capacity nodes (setting their splitting-edge capacity to zero) with reachability-only checks for unconstrained nodes.

1.4.6 Min-cut family and global connectivity

Beyond a single representative minimum cut, the full minimum-cut family captures all alternative equivalent bottleneck structures. The lattice structure due to Picard and Queyranne (1982) characterises this family efficiently. After an optimal solve, the *free zone* $F_{\text{free}} = S^{**} \setminus S^*$ consists of nodes not assigned to either the source or sink side of every minimum cut. All minimum cuts correspond to partitions $S = S^* \cup R$ where $R \subseteq F_{\text{free}}$, giving exactly $2^{|F_{\text{free}}|}$ minimum cuts in the worst case. In implementation, when $|F_{\text{free}}| \leq 62$ the exact count is computed by bit-shift; larger free zones are truncated to a user-specified cut limit and an incompleteness flag is returned. Each enumerated cut is validated by capacity equality against the solved max-flow value within numerical tolerance.

This matters for reliability because different physical component sets may represent equally critical vulnerability barriers. Enumerating the family identifies all minimal failure mode candidates for the zero-throughput top event without requiring per-cut re-solves after the baseline.

Global connectivity metrics are derived from repeated flow constructions using unit-capacity networks:

- **Edge connectivity** λ : unit-capacity graph with super-sink aggregation, one solve per source node; integer-valued output verified by integrality check.
- **Node connectivity** κ : node-split graph with unit node capacities, one solve per source node; integer-valued output verified.

Under unit-capacity constructions, the integrality theorem ensures that λ and κ are exact integer counts of edge-disjoint and internally node-disjoint path families respectively, consistent with Menger’s path-cut equalities.

1.5 Exactness, Validation, and Scope

1.5.1 Exactness definition used in this chapter

In this chapter, *exactness* means exact deterministic optimality for the formulated optimization problem instance. Returned flow values are the global optimum produced

by the selected exact combinatorial algorithm under IEEE-754 arithmetic, not a statistical estimate and not a heuristic approximation. Numerical tolerance appears only in floating-point equality predicates used for post-solve consistency checks (for example, verifying that saturation conditions and conservation balances hold within machine-precision bounds); it is not used as an optimization convergence criterion replacing exact termination. This exactness definition is distinct from global combinatorial exhaustiveness: where candidate-set pruning is applied, exactness describes results on evaluated candidates, not globally across all possible edge subsets or perturbation scenarios.

1.5.2 Post-solve validation

Every public solver call with validation enabled applies three post-solve consistency checks before downstream analyses are interpreted:

1. **Capacity feasibility:** all edge flows satisfy $0 \leq f(u, v) \leq c(u, v)$.
2. **Node-wise conservation:** $\sum_{u:(u,v) \in E} f(u, v) = \sum_{w:(v,w) \in E} f(v, w)$ at all internal nodes.
3. **Max-flow/min-cut consistency:** the solved flow value and the solved cut capacity agree within numerical tolerance.

Together these provide primal feasibility and dual optimality consistency before derived analyses are interpreted. For flow decomposition outputs, an additional decomposition consistency check verifies that the sum of path flows exactly reconstructs the solved edge-flow mapping. These checks are theorem-backed rather than heuristic: the max-flow/min-cut equality in check (3) follows from the Ford–Fulkerson theorem, so a failure indicates a code or input error, not an acceptable approximation gap.

1.5.3 Mandatory scope qualifiers

Three scope boundaries are explicit throughout this chapter and must accompany any result reporting:

- **Candidate-pruned perturbation scope:** sensitivity and failure-impact analyses use candidate sets drawn from baseline min-cut structure. Results are exact on evaluated candidates. Global exhaustiveness over all edges or all combinatorial subsets requires explicit full-space evaluation and should not be assumed from candidate-pruned results.
- **No probabilistic uncertainty propagation:** this chapter does not compute $P(\text{performance} \geq T)$ or any probability-weighted reliability index. Uncertainty

models for random capacity failures and component failure probabilities are deferred to subsequent chapters that consume the structural outputs generated here.

- **No dynamic flow adaptation:** all capacities and network topology are static within each solve. Time-varying degradation, adaptive rerouting, and multi-period operational models are outside the scope of this deterministic layer.

These qualifiers are design-scope declarations, not deficiencies. The deterministic structural layer is non-redundant because probabilistic models require a correct structural map of bottlenecks, failure modes, and capacity margins before event probabilities can be assigned and propagated meaningfully.

1.6 Reliability Engineering Positioning

1.6.1 Structural pre-probabilistic role

This methodology is best understood as the structural precursor to probabilistic reliability analysis. Binary reliability models map component states to Boolean system success or failure, evaluating whether connectivity is maintained. Capacity analysis instead maps component capacities to a continuous performance function $F(c) = \text{max-flow}$, which can remain strictly positive while falling below operational thresholds (Ahuja, Magnanti, and Orlin, 1993). Binary connectivity can remain true while throughput degrades substantially; a water network, power transmission corridor, or communication backbone can remain connected in a topological sense while delivering service well below demand. Capacity analysis quantifies this performance-degradation dimension that binary models cannot capture.

The outputs of this deterministic layer — throughput baselines, min-cut sets, SPOF candidates, edge-level threshold margins, and connectivity indicators — define the event structure and component-level state thresholds for subsequent uncertainty propagation. Without this structural foundation, probabilistic propagation can become disconnected from the actual transfer constraints that govern service-level performance. This chapter therefore occupies a necessary and non-redundant position in the thesis architecture.

1.6.2 Birnbaum-style importance analogy

Classical Birnbaum importance in reliability theory is defined as the marginal sensitivity of system reliability to a component’s reliability parameter: $I_B(i) = \partial h(\mathbf{p}) / \partial p_i$, where h is the system reliability function and p_i is component i ’s reliability (Birnbaum, 1969). The present framework uses a structurally analogous measure in flow space: component

influence is quantified by the range of system throughput over the component’s full capacity range, $I_B(e) = F(c_e \leftarrow \infty) - F(c_e \leftarrow 0)$. Both measures map component-level parameters to system-level response and support component ranking by system influence.

The analogy is conceptually useful for interpretation and for contextualising these structural metrics within the classical reliability-importance literature. However, it should not be represented as formal probabilistic equivalence: the response variable in the flow analogue is deterministic throughput, not probability of binary success. The flow-analogue importance score $I_B(e)$ is a structural sensitivity measure on a continuous performance function, evaluated without reference to failure probabilities or component reliability distributions.

1.6.3 Minimum cuts as structural failure-mode carriers

For top events defined as complete throughput loss, minimum cuts represent minimal structural separating sets: each minimum cut is a set of edges whose simultaneous removal reduces max-flow to zero, and no proper subset has this property. These sets are therefore the minimal failure modes of the zero-throughput top event and map directly to physical barrier configurations in infrastructure terms.

For nonzero service targets, threshold-based failure events generalise this concept. A capacity configuration fails the target T when $F(c) < T$, which is not equivalent to zero flow and is not necessarily caused by a minimum cut. The relevant failure boundary for target T is characterised by threshold margins $M_e = c(e) - c_e^*(\text{degrade})$ for each component: the system is at risk when any critical edge’s margin approaches zero. This distinction is practically important in infrastructure planning, where regulatory or contractual service thresholds define the true reliability boundary rather than complete disconnection.

These deterministic structural outputs — minimum-cut sets, SPOF candidates, and threshold margins — constitute the foundation passed forward to subsequent probabilistic chapters, where failure probabilities are assigned to components and propagated through the structural map established here.

1.7 Computational Complexity Summary

Let $|V| = n$ and $|E| = m$. Complexity is reported as baseline per-solve solver complexity multiplied by the number of required solves for each derived analysis task. Throughput and bottleneck extraction are therefore single-solve tasks, while connectivity and combinatorial failure studies are repeated-solve tasks whose scaling is dominated by solve count. The directed global min-cut incurs an $O(n)$ multiplier on per-solve cost. Sensitivity and failure-impact perturbation studies are not free re-solves: each perturbed-network evaluation is a full max-flow call, and cost scales linearly with the number of tested candidates

or combinations.

Task	Solve Pattern	Scaling Driver
Baseline max-flow/min-cut	1 solve	chosen max-flow algorithm
Min-cut lattice extraction	0 additional solves	residual BFS after baseline
SPOF edge identification	0 additional solves	lattice reachability after baseline
Flow decomposition	0 additional solves	iterative path extraction
Single-edge failure impact	up to m solves	number of tested candidate edges
k -edge failure impact	up to $\binom{m}{k}$ solves	candidate combinations (bounded by 1)
Capacity degradation scenarios	1 solve per scenario	number of explicit scenarios
Threshold analysis per edge	boundary + recursion solves	recursion depth per edge
Edge connectivity λ	n unit-capacity solves	source-set size
Node connectivity κ	n node-capacitated solves	source-set size on split graph
Directed global min-cut	$2(n - 1)$ solves	terminal-pair call count
Structural path enumeration	0 solver calls	graph traversal (path-limit bounded)

Table 1.1: Complexity viewed as per-solve cost multiplied by solve count per derived analysis task.

This presentation avoids over-promising symbolic complexity in isolation and matches operational runtime behaviour in integrated reliability workflows.

1.8 Implementation and Software Notes

1.8.1 Implementation rationale

The framework is implemented in Julia, using sparse edge-key dictionaries (`Dict{Tuple{Int64, Int64}}`), for capacity and flow storage and adjacency index dictionaries (`Dict{Int64, Set{Int64}}`) for breadth-first and depth-first traversal. This aligns directly with upstream input processing and retains edge-level semantic identity through all perturbation, decomposition, and structural analyses. Typed result structures (`FlowSolveResult` and analysis-specific result types) centralise module outputs, improve type stability, and carry the residual state, flow map, cut partition, and saturation set needed by downstream analysis modules.

The DAG invariant is guaranteed by the upstream `InputProcessingModule` and treated as a contract in this layer rather than re-checked at each analysis call, preventing duplicate validation overhead. The `Int64` guard on super-node index construction (using $\min(V) - 1$ and $\min(V) - 2$) protects against pathological index-space cases in which node identifiers approach the minimum representable 64-bit integer, ensuring correctness across arbitrary node-ID conventions from input preprocessing.

1.8.2 Positioning relative to generic graph-flow packages

The Julia package `GraphsFlows.jl` provides standard max-flow and min-cut routines for single-source single-sink formulations. For a single one-off source-sink max-flow, such packages solve an equivalent optimization problem with comparable theoretical complexity classes (Ahuja, Magnanti, and Orlin, 1993). No unconditional runtime superiority claim is made without controlled benchmarking across matched graph classes and hardware conditions.

The distinct contribution of this framework is integration of multi-terminal handling with a downstream reliability-oriented analysis pipeline over a shared domain-specific data model. Specific additions include: a native multi-source multi-sink API with exact super-node reduction; rich typed result structures carrying residual state and cut artifacts for downstream use; integrated modules for flow decomposition, parametric threshold analysis, node-capacitated flow, min-cut lattice enumeration, global connectivity, and failure-impact and sensitivity workflows; and post-solve validation throughput. This methodological breadth and pipeline integration, rather than raw single-solve throughput, is the primary rationale for maintaining a custom framework alongside the general-purpose ecosystem.

1.9 Chapter Summary

This chapter has established a deterministic capacity-flow methodology for infrastructure reliability analysis. The formal capacitated directed network model was defined with source and sink sets, flow conservation constraints, and the DAG-as-upstream-invariant design contract between preprocessing and analysis layers. Max-flow/min-cut duality was established as the central theoretical result (Ford and Fulkerson, 1956), providing simultaneous computation of throughput and binding bottleneck from a single optimal solve. Supporting theorems — integrality (Ford and Fulkerson, 1962), Menger edge and node forms (Menger, 1927), the Picard–Queyranne lattice structure (Picard and Queyranne, 1982), node-splitting equivalence, and flow decomposition — were presented and connected to specific computational modules and reliability interpretations. Three implemented solver families (Edmonds–Karp, Dinic as implementation default, and push-relabel) were described under a shared residual-graph contract, with the Dinic default framed explicitly as an implementation design choice rather than a universal performance claim.

Derived analyses were presented for sensitivity (three explicit metrics with candidate-set scoping), discrete failure impact (single-edge and k -edge with combination bounds), single-point-of-failure detection via the Picard–Queyranne lattice, flow decomposition versus structural path enumeration, parametric threshold margins (degradation and upgrade

with piecewise-linear analysis), node-capacitated flow via exact node splitting, min-cut family enumeration, and global connectivity. Exactness was defined precisely as exact deterministic optimality for each solved instance, with numerical tolerance confined to floating-point equality predicates in post-solve primal-feasibility and dual-consistency checks. Three mandatory scope boundaries were stated explicitly: candidate-pruned perturbation exactness is scoped to evaluated candidates; no probabilistic uncertainty propagation is computed in this layer; and no dynamic or time-varying flow adaptation is modelled.

The resulting structural reliability map — baseline throughput, binding bottlenecks, SPOF sets, min-cut failure-mode families, threshold margins, and connectivity indicators — constitutes the required structural foundation for the next chapter, where uncertainty models and probabilistic propagation are introduced on top of the deterministic structural layer developed here. This chapter provides the pre-probabilistic scaffold that ensures probabilistic modelling is grounded in a validated structural understanding of system capacity constraints.

Bibliography

- [1] Ahuja, R.K., Magnanti, T.L., Orlin, J.B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs, NJ.
- [2] Birnbaum, Z.W. (1969). On the importance of different components in a multicomponent system. In P.R. Krishnaiah (Ed.), *Multivariate Analysis II*, pp. 581–592. Academic Press, New York. DOI: 10.21236/AD0670563.
- [3] Dinic, E.A. (1970). Algorithm for solution of a problem of maximum flow in a network with power estimation. *Soviet Mathematics Doklady*, 11, 1277–1280.
- [4] Edmonds, J., Karp, R.M. (1972). Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2), 248–264. DOI: 10.1145/321694.321699.
- [5] Ford, L.R., Fulkerson, D.R. (1956). Maximal flow through a network. *Canadian Journal of Mathematics*, 8, 399–404. DOI: 10.4153/CJM-1956-045-5.
- [6] Ford, L.R., Fulkerson, D.R. (1962). *Flows in Networks*. Princeton University Press, Princeton, NJ.
- [7] Gallo, G., Grigoriadis, M.D., Tarjan, R.E. (1989). A fast parametric maximum flow algorithm and applications. *SIAM Journal on Computing*, 18(1), 30–55. DOI: 10.1137/0218003.
- [8] Goldberg, A.V., Tarjan, R.E. (1988). A new approach to the maximum-flow problem. *Journal of the ACM*, 35(4), 921–940. DOI: 10.1145/48014.61051.
- [9] Hao, J.X., Orlin, J.B. (1994). A faster algorithm for finding the minimum cut in a directed graph. *Journal of Algorithms*, 17(3), 424–446. DOI: 10.1006/jagm.1994.1043.
- [10] Menger, K. (1927). Zur allgemeinen Kurventheorie. *Fundamenta Mathematicae*, 10, 96–115. DOI: 10.4064/fm-10-1-96-115.
- [11] Picard, J.-C., Queyranne, M. (1982). A network flow solution to some nonlinear 0–1 programming problems, with applications to graph theory. *Networks*, 12(2), 141–159. DOI: 10.1002/net.3230120206.

- [12] Whitney, H. (1932). Congruent graphs and the connectivity of graphs. *American Journal of Mathematics*, 54(1), 150–168. DOI: 10.2307/2371086.